2026-7-5 Linux开发工具--代码编译器

- 1. gcc 与程序翻译
  - 1.1 编译器在开发流程中的位置
    - 1.1.1 编译器负责什么
    - 1.1.2 CPU 为什么需要机器指令
    - 1.1.3 一次编译不一定要经过所有显式命令
  - 1.2 gcc 与 g++ 的区别
    - 1.2.1 基本定位
    - 1.2.2 为什么不自动链接 C++ 标准库会出问题
    - 1.2.3 gcc 与 g++ 的选项关系
- 2. gcc 的四个编译阶段
  - 2.1 为什么要把编译过程拆成四段
    - 2.1.1 每一段都有明确产物
    - 2.1.2 为什么拆开更有助于理解
  - 2.2 预处理
    - 2.2.1 预处理的基本工作
    - 2.2.2 头文件展开
    - 2.2.3 宏替换
    - 2.2.4 条件编译
    - 2.2.5 头文件保护
    - 2.2.6 去除注释
  - 2.3 编译
    - 2.3.1 从 C 到汇编
    - 2.3.2 为什么中间会出现汇编
  - 2.4 汇编
    - 2.4.1 从汇编到可重定位目标文件
    - 2.4.2 .o 中有什么
    - 2.4.3 为什么 .o 不能直接运行
  - 2.5 链接
    - 2.5.1 生成可执行文件
    - 2.5.2 默认输出文件
  - 2.6 四阶段命令汇总
    - 2.6.1 分步执行
    - 2.6.2 一步完成
    - 2.6.3 选项对比
- 3. 从机器码理解编译器

- 3.1 二进制、指令集与程序
  - 3.1.1 CPU 为什么执行二进制
  - 3.1.2 翻译语言的本质
- 3.2 从纸带到高级语言
  - 3.2.1 早期编程方式
  - 3.2.2 汇编语言
  - 3.2.3 高级语言与编译器
- 3.3 编译器自举
  - 3.3.1 先有语言还是先有编译器
  - 3.3.2 自举的基本过程
- 3.4 一门语言需要什么工具
  - 3.4.1 三个必要条件
  - 3.4.2 三者分别做什么
- 4. 库、静态链接与动态链接
  - 4.1 为什么需要链接
    - 4.1.1 函数声明和函数实现
    - 4.1.2 多文件程序
  - 4.2 库是什么
    - 4.2.1 库是方法与对象的集合
    - 4.2.2 头文件与库的分工
  - 4.3 动态链接
    - 4.3.1 动态链接的基本思想
    - 4.3.2 课堂中的地址理解
    - 4.3.3 动态链接的优点
    - 4.3.4 动态链接的代价
  - 4.4 静态链接
    - 4.4.1 静态链接的基本思想
    - 4.4.2 静态链接的优点
    - 4.4.3 静态链接的缺点
  - 4.5 静态库与动态库
    - 4.5.1 Linux 常见库后缀
    - 4.5.2 .o 不是库
    - 4.5.3 gcc 默认链接方式
    - 4.5.4 ldd 查看动态库依赖
    - 4.5.5 动态库文件名
  - 4.6 强制静态链接
    - 4.6.1 使用 -static

        - 4.6.2 为什么可能失败
        - 4.6.3 比较文件大小
        - 4.6.4 验证静态链接结果
    - 4.7 静态链接与动态链接总结
- 5. gcc 常用选项
    - 5.1 阶段控制
        - 5.1.1 预处理、编译、汇编
        - 5.1.2 输出文件
    - 5.2 调试、警告与优化
        - 5.2.1 调试信息
        - 5.2.2 显示警告
        - 5.2.3 优化级别
    - 5.3 链接选项
        - 5.3.1 静态链接
        - 5.3.2 生成共享库
        - 5.3.3 链接指定库
        - 5.3.4 增加头文件目录
    - 5.4 常用选项总表
- 6. gcc 实战命令
    - 6.1 单文件一步编译
        - 6.1.1 最简命令
        - 6.1.2 开发阶段常用
    - 6.2 单文件分阶段编译
    - 6.3 多文件编译
        - 6.3.1 分别生成目标文件
        - 6.3.2 链接
        - 6.3.3 一次完成
    - 6.4 验证可执行文件
        - 6.4.1 查看文件格式
        - 6.4.2 查看动态依赖
        - 6.4.3 生成并对比静态版本
    - 6.5 C++ 程序
- 7. 常见错误与排查
    - 7.1 预处理错误
        - 7.1.1 找不到头文件
        - 7.1.2 宏未定义
    - 7.2 编译错误

        - 7.2.1 语法错误
        - 7.2.2 类型错误
    - 7.3 汇编错误
    - 7.4 链接错误
        - 7.4.1 undefined reference
        - 7.4.2 编译通过但链接失败
        - 7.4.3 缺少静态库
    - 7.5 运行错误
        - 7.5.1 找不到动态库
        - 7.5.2 .o 不能直接运行
        - 7.5.3 架构不匹配
    - 7.6 排查顺序
- 8. gcc 命令速查与练习
    - 8.1 四阶段速查
    - 8.2 链接与库速查
    - 8.3 常用选项速查
    - 8.4 建议练习
        - 8.4.1 观察四个阶段
        - 8.4.2 比较链接方式
        - 8.4.3 观察头文件保护
        - 8.4.4 制造链接错误
    - 8.5 gcc 学习主线

1. gcc 与程序翻译

1.1 编译器在开发流程中的位置

1.1.1 编译器负责什么

编辑器负责写出源程序，编译器负责把源程序翻译成计算机能够执行的代码。

可以先用一条最简单的主线理解：

```
源代码
→ 预处理
→ 编译
→ 汇编
```

```
→ 链接
→ 可执行程序
```

gcc 是 GNU 编译器集合的驱动程序。调用 gcc 时，它会根据文件类型和选项启动预处理、编译、汇编和链接等不同环节。

1.1.2 CPU 为什么需要机器指令

CPU 最终能够执行的是机器指令。高级语言、汇编语言和机器指令之间可以理解为逐层映射：

```
C/C++ 源程序
    ↓ 编译器翻译
汇编语言
    ↓ 汇编器
机器指令
    ↓
CPU 执行
```

所谓“翻译程序”，本质是把程序员容易表达的源程序转换成 CPU 能识别的指令序列。

1.1.3 一次编译不一定要经过所有显式命令

执行：

```
gcc hello.c -o hello
```

gcc 会自动完成预处理、编译、汇编和链接。学习时把四个阶段拆开，是为了观察每一步的输入、输出和职责。

1.2 gcc 与 g++ 的区别

1.2.1 基本定位

工具	主要用途	默认行为
`gcc`	C 语言编译	`.c` 文件按 C 规则编译，不自动链接 C++ 标准库

工具	主要用途	默认行为
`g++`	C++ 语言编译	`.cpp` 等文件按 C++ 规则编译，并链接 C++ 标准库

一个常见误区是认为 gcc“只能编译 C”，g++“只能编译 C++”。实际上二者都基于 GCC 后端，能处理多种文件类型，但驱动方式、语言规则和默认链接库不同。

对刚开始学习 Linux 的代码，推荐：

```
C 程序 → gcc
C++ 程序 → g++
```

这样更符合语言规则，也能避免忘记链接 C++ 标准库。

1.2.2 为什么不自动链接 C++ 标准库会出问题

C++ 程序可能使用字符串、异常、标准输入输出流等特性，这些实现通常位于 C++ 标准库中。

如果使用不合适的编译驱动，可能出现：

```
undefined reference to std::cout
undefined reference to operator new
```

这通常不是源代码真的没有写对，而是编译驱动没有把 C++ 运行时库正确加入链接过程。

1.2.3 gcc 与 g++ 的选项关系

gcc 和 g++ 在预处理、编译、汇编、链接方面的大部分选项是共通的，例如：

```
-E  -S  -c  -o  -g  -static  -O1  -O2  -Wall
```

需要重点区分的是：

- 语言标准。
- 默认头文件搜索行为。

- 是否自动链接 C++ 标准库。
- 对 C++ 支持的默认链接方式。

2. gcc 的四个编译阶段

2.1 为什么要把编译过程拆成四段

2.1.1 每一段都有明确产物

```
hello.c
  ↓ 预处理
hello.i
  ↓ 编译
hello.s
  ↓ 汇编
hello.o
  ↓ 链接
hello
```

阶段	输入	输出	gcc 选项
预处理	`.c` 等源文件	`.i` 或预处理结果	`-E`
编译	预处理后的源文件	`.s` 汇编文件	`-S`
汇编	`.s` 汇编文件	`.o` 目标文件	`-c`
链接	`.o` 文件和库	可执行文件或库	无对应停止选项

2.1.2 为什么拆开更有助于理解

把阶段拆开以后，可以准确判断错误发生在哪一步：

- 预处理找不到头文件。
- 编译阶段发现语法错误。
- 汇编阶段有问题。
- 链接阶段找不到函数实现或库。

这比把所有问题都笼统地称为“gcc 编译失败”更准确。

2.2 预处理

2.2.1 预处理的基本工作

预处理主要处理以 `#` 开头的预处理指令，并完成：

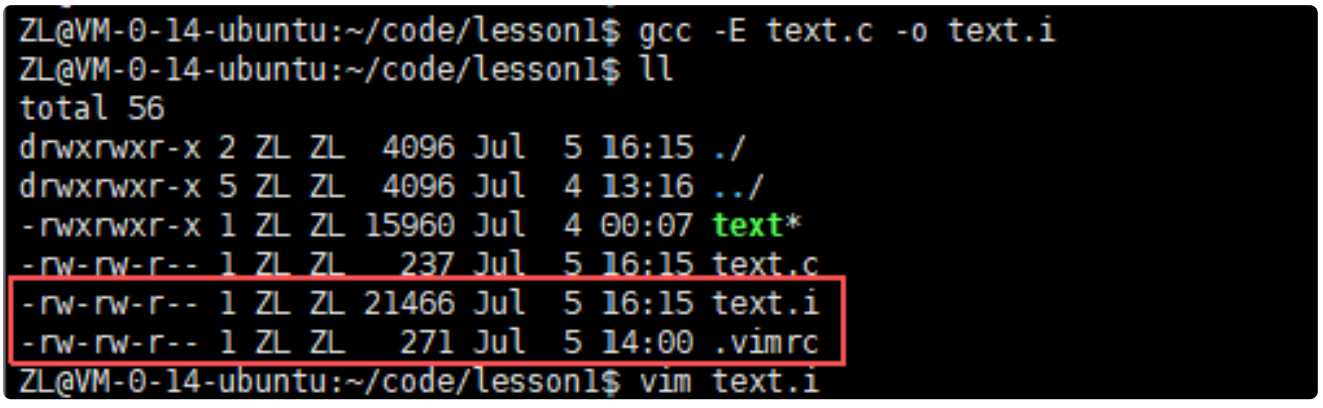
1. 头文件展开。
2. 宏替换。
3. 条件编译。
4. 去除注释。

命令：

```
gcc -E hello.c -o hello.i
```

`-E` 表示只执行预处理，然后停止。

`-o hello.i` 表示把预处理结果写入 `hello.i`。如果不写 `-o`，预处理结果通常会输出到标准输出。



2.2.2 头文件展开

源代码中：

```
#include <stdio.h>
```

经过预处理后，`stdio.h` 的内容会被展开到源文件中。

头文件通常保存：

- 类型声明。
- 函数声明。
- 宏定义。
- 结构体声明。

真正的函数实现可能在库文件中。把声明和实现分开，可以避免重复保存代码，也方便多个源文件共享同一套接口。

2.2.3 宏替换

预处理会把宏定义替换到使用位置，例如：

```
#define MAX_SIZE 1024
int buffer[MAX_SIZE];
```

预处理后，`MAX_SIZE` 会出现为对应的值。

宏替换是纯文本层面的展开，不等同于运行时变量。

2.2.4 条件编译

条件编译可以根据宏或环境选择不同代码：

```
#ifdef DEBUG
    printf("debug mode\n");
#endif
```

它的本质是对代码进行裁剪或选择。不同场景可以使用：

- `#ifdef` / `#ifndef`
- `#if` / `#elif` / `#else`
- `#endif`

条件编译可用于区分调试和发布版本、支持不同平台、保留特定功能或隐藏不需要的接口。

2.2.5 头文件保护

一个源程序可能通过多条路径重复包含同一个头文件。为了防止重复声明，常使用头文件保护：

```
#ifndef MY_HEADER_H
#define MY_HEADER_H

/* 声明 */

#endif
```

第一次包含时，`MY_HEADER_H` 尚未定义，头文件内容生效；再次包含时，条件判断失败，内容不会重复展开。

也可以使用：

```
#pragma once
```

现代编译器通常都支持这种更简洁的头文件保护方式。

2.2.6 去除注释

预处理阶段会去掉源文件中的注释。注释用于给人阅读，不参与最终机器程序。

2.3 编译

2.3.1 从 C 到汇编

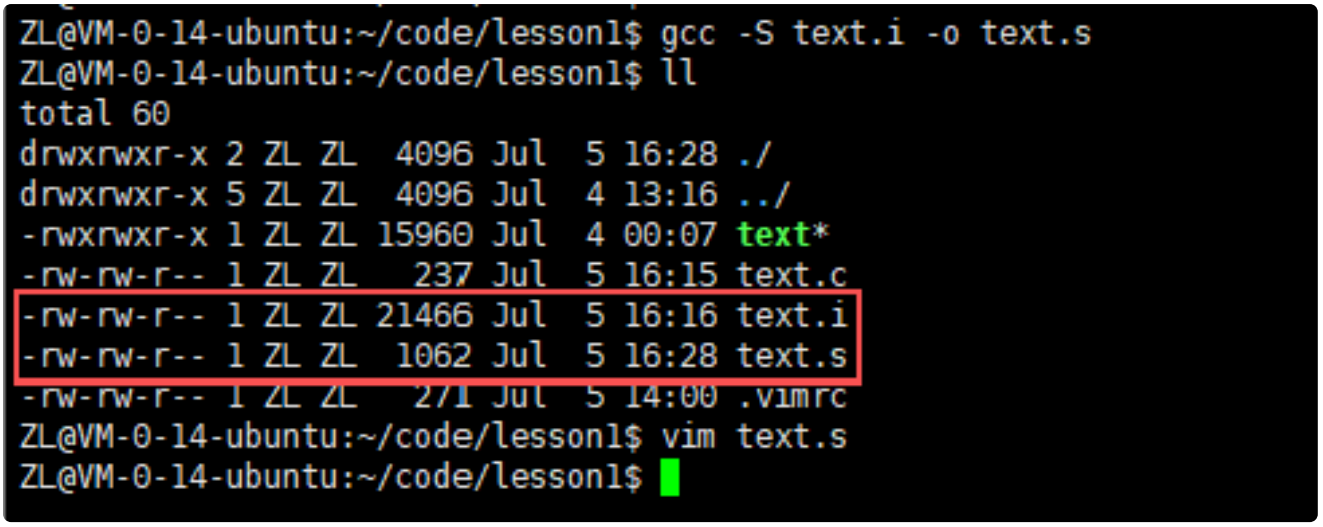
命令：

```
gcc -S hello.i -o hello.s
```

`-S` 表示预处理和编译完成后停止，不继续汇编。

编译阶段会：

1. 检查语法和基本语义。
2. 分析类型和结构。
3. 进行必要的转换和优化。
4. 生成对应平台的汇编代码。



如果源代码存在语法错误，通常会在这个阶段报错。

2.3.2 为什么中间会出现汇编

高级语言更适合人表达，机器指令适合 CPU 执行。汇编语言处在中间：

- 它比二进制机器码更容易阅读。
- 它的助记符能与机器指令一一对应。
- 编译器可以先输出汇编，再由汇编器转换为机器码。

汇编文件不是最终程序，它仍需要通过汇编器转换成目标文件。

2.4 汇编

2.4.1 从汇编到可重定位目标文件

命令：

```
gcc -c hello.s -o hello.o
```

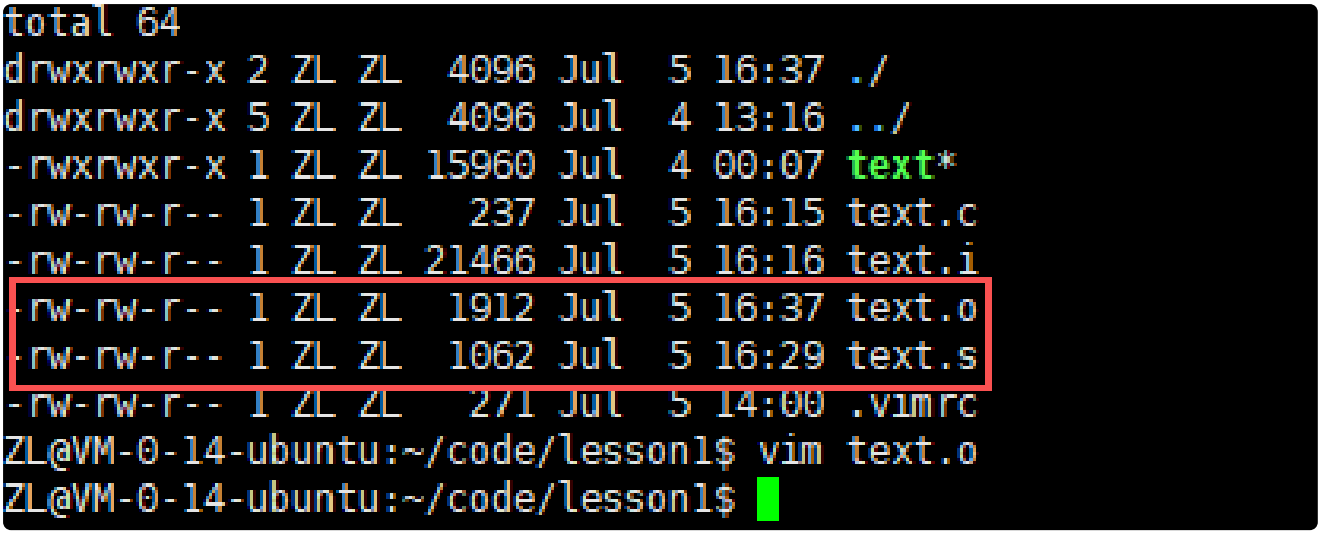
更常用的写法是直接从源文件生成目标文件：

```
gcc -c hello.c
```

这会生成：

```
hello.o
```

`-c` 表示生成目标文件后停止，不执行最终链接。



2.4.2 `.o` 中有什么

`.o` 是目标文件，通常包含：

- 已经翻译好的机器代码。
- 数据段。
- 符号表。
- 尚未最终确定的地址和重定位信息。
- 尚未解析的外部函数引用。

因此它是“可重定位目标文件”，不是完整的可执行程序。

2.4.3 为什么 `.o` 不能直接运行

尝试：

```
./hello.o
```

可能出现：

```
bash: ./hello.o: cannot execute binary file
```

原因不是 `.o` 完全没有机器码，而是它还没有经过链接：

- 外部函数地址可能尚未确定。
- 程序入口和多个目标文件的组合关系尚未完成。
- 运行库还未正确加入。

例如程序调用了 `printf`，头文件只提供声明，真正实现位于 C 运行库中。必须经过链接，才能把程序和所需库连接起来。

2.5 链接

2.5.1 生成可执行文件

```
gcc hello.o -o hello
```

链接阶段的主要工作：

1. 合并多个目标文件。
2. 解析函数和全局变量引用。
3. 加入启动代码。
4. 链接所需的运行库。
5. 生成可执行文件。

```
ZL@VM-0-14-ubuntu:~/code/lesson1$ gcc text.o -o text
ZL@VM-0-14-ubuntu:~/code/lesson1$ ll
total 64
drwxrwxr-x 2 ZL ZL  4096 Jul  5 16:40 ./
drwxrwxr-x 5 ZL ZL  4096 Jul  4 13:16 ../
-rwxrwxr-x 1 ZL ZL 15960 Jul  5 16:40 text*
-rw-rw-r-- 1 ZL ZL   237 Jul  5 16:15 text.c
-rw-rw-r-- 1 ZL ZL 21466 Jul  5 16:16 text.i
-rw-rw-r-- 1 ZL ZL  1913 Jul  5 16:37 text.o
-rw-rw-r-- 1 ZL ZL  1062 Jul  5 16:29 text.s
-rw-rw-r-- 1 ZL ZL   271 Jul  5 14:00 .vimrc
ZL@VM-0-14-ubuntu:~/code/lesson1$ .\text
.text: command not found
ZL@VM-0-14-ubuntu:~/code/lesson1$ ./text
Hello World
Hello World
Hello World
Hello WWorldorld
Hello wooroorr
Hello World
Hello World
```

执行：

```
./hello
```

运行生成的可执行程序。

2.5.2 默认输出文件

如果不指定 `-o`：

```
gcc hello.c
```

通常会生成：

```
a.out
```

`-o` 用于明确指定输出名称：

```
gcc hello.c -o hello
```

2.6 四阶段命令汇总

2.6.1 分步执行

```
gcc -E hello.c -o hello.i
gcc -S hello.i -o hello.s
gcc -c hello.s -o hello.o
gcc hello.o -o hello
./hello
```

2.6.2 一步完成

```
gcc hello.c -o hello
```

2.6.3 选项对比

选项	执行到哪一步停止	阶段包括
`-E`	预处理结束	预处理
`-S`	编译结束	预处理、编译
`-c`	汇编结束	预处理、编译、汇编
无	完整链接	预处理、编译、汇编、链接

3. 从机器码理解编译器

3.1 二进制、指令集与程序

3.1.1 CPU 为什么执行二进制

计算机内部使用二进制表示数据。CPU 的指令集定义了它能识别的机器指令格式：

```
操作码
操作数
寄存器
内存地址
```

把多条机器指令组合起来，就形成程序。

3.1.2 翻译语言的本质

源程序不能直接交给普通 CPU 执行。编译器的核心任务，是把高级语言构建出的语义转换成目标 CPU 的指令集。

3.2 从纸带到高级语言

3.2.1 早期编程方式

在高级语言出现之前，程序员需要直接处理机器码或汇编。早期纸带、打孔等方式就是用物理形式记录指令和数据。



3.2.2 汇编语言

汇编语言用助记符代替机器码，例如：

```
mov
add
jmp
```

这样更容易让人理解，但最终仍然需要汇编器将助记符转换为机器指令。

3.2.3 高级语言与编译器

C 语言出现后，程序员可以用更接近人类表达的结构编写程序，再由编译器转换为汇编和机器码。

因此，语言和编译器是相伴发展的：

```
新的语言表达
→ 需要新的语法规则
→ 需要配套编译器
→ 编译器生成目标机器指令
```

3.3 编译器自举

3.3.1 先有语言还是先有编译器

现代编译器通常用高级语言编写。例如一个 C 编译器可能主要用 C 编写。

这看起来像“鸡生蛋、蛋生鸡”的问题：

- 没有编译器，怎样编译编译器源码？
- 没有编译器源码，怎样生成编译器？

3.3.2 自举的基本过程

解决方式是逐层建立：

```
用机器码手工编写最小编译器
→ 用它编译汇编器或早期编译器
→ 用早期编译器构建更完整的语言工具
→ 用新编译器重新编译自己的源代码
```

当新编译器能够正确编译自身代码，并生成可工作的新版本时，就称为编译自举。

3.4 一门语言需要什么工具

3.4.1 三个必要条件

开发或使用一门语言通常需要：

1. 编译器。
2. 头文件。
3. 库文件。

3.4.2 三者分别做什么

组件	作用
编译器	把源代码翻译为目标代码
头文件	向编译器提供类型和接口声明
库文件	提供函数和对象的实际实现

安装“C/C++ 开发环境”时，往往不只是安装 gcc，还要安装标准库、头文件和构建工具。只安装编译器而没有相应头文件或库，编译和链接仍会失败。

4. 库、静态链接与动态链接

4.1 为什么需要链接

4.1.1 函数声明和函数实现

例如使用：

```
#include <stdio.h>
printf("hello\n");
```

头文件告诉编译器 `printf` 的参数和返回类型，但 `<stdio.h>` 通常不包含 `printf` 的完整函数体。

函数实现位于运行库中。链接器负责把目标文件对 `printf` 的引用连接到库中的实现。

4.1.2 多文件程序

项目可能有：

```
main.c
math.c
io.c
```

各自编译：

```
gcc -c main.c
gcc -c math.c
gcc -c io.c
```

生成：

```
main.o
math.o
io.o
```

再链接：

```
gcc main.o math.o io.o -o app
```

链接器把多个目标文件组合成一个程序，并解析它们之间的函数和变量引用。

4.2 库是什么

4.2.1 库是方法与对象的集合

库可以理解为一组已经编译好的函数、对象和资源的集合。程序不需要重新实现所有通用功能，只需要调用库提供的接口。

常见例子：

- C 标准库。
- C++ 标准库。
- 数学库。
- 网络库。
- 图形库。

4.2.2 头文件与库的分工

```
头文件 → 编译时告诉编译器“有什么”
库文件 → 链接或运行时提供“怎么实现”
```

编译器需要知道函数声明，链接器需要找到函数实现。

4.3 动态链接

4.3.1 动态链接的基本思想

动态链接不会把库的全部方法复制进可执行文件，而是在程序启动或运行时加载共享库。

结构可以理解为：

```
可执行程序
  → 依赖关系
  → 动态库
  → 调用库中的函数
```

4.3.2 课堂中的地址理解

动态链接建立的是程序和共享库之间的关联。程序运行时，动态加载器找到库，并把函数地址连接起来。

可以用“问路”类比：

- 程序知道需要某个函数。
- 程序中保存的不是函数代码本身，而是寻找和跳转到该函数的关联信息。
- 动态库缺失或版本不匹配时，程序可能无法启动。

4.3.3 动态链接的优点

- 可执行文件通常更小。
- 多个程序可以共享同一个库。
- 符合兼容条件时，库升级后程序不必重新链接。
- 系统可以集中维护公共运行库。

4.3.4 动态链接的代价

- 运行时必须能找到相应动态库。
- 动态库版本或路径不匹配会有运行风险。
- 程序启动和符号解析存在额外开销。

4.4 静态链接

4.4.1 静态链接的基本思想

静态链接会把程序实际需要的库代码复制或合并进最终可执行文件。

```
目标文件 + 静态库
→ 链接器提取所需代码
→ 生成包含库代码的可执行文件
```

4.4.2 静态链接的优点

- 部署时对目标系统静态库的运行时依赖较少。
- 库代码已经在可执行文件中。
- 某些情况下可以减少函数跳转和运行时库解析。

4.4.3 静态链接的缺点

- 可执行文件通常更大。
- 多个程序依赖同一库时，磁盘上可能保存多份库代码。
- 库更新后，需要重新链接相关程序。
- 现代内存管理器会使用写时复制等机制优化物理页共享，但每个可执行文件仍携带自己的库代码副本。

4.5 静态库与动态库

4.5.1 Linux 常见库后缀

类型	Linux 后缀	典型名称
静态库	`.a`	`libxxx.a`
动态库、共享库	`.so`	`libxxx.so`

Windows 中类似概念常使用：

```
静态库：.lib
动态库：.dll
```

4.5.2 `.o` 不是库

区别如下：

```
.o   单个目标文件
.a   多个目标文件的静态归档
.so 共享库
```

静态库并不是一个可以直接运行的程序，它是链接器可使用的目标代码集合。

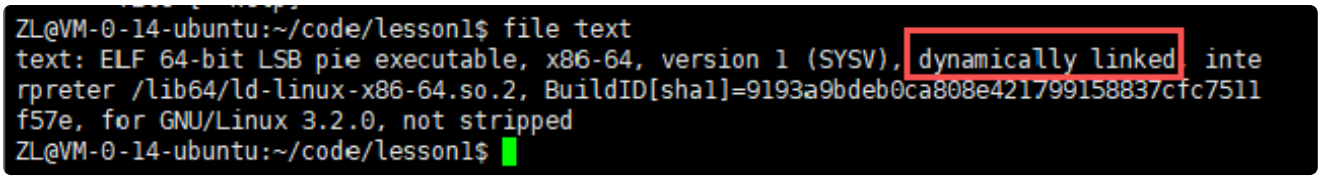
4.5.3 gcc 默认链接方式

在常见的桌面和服务器 Linux 环境，gcc 默认生成动态链接程序：

```
gcc hello.c -o hello
```

可以通过 `file` 验证：

```
file hello
```



输出中可能出现：

```
ELF 64-bit LSB executable
dynamically linked
```

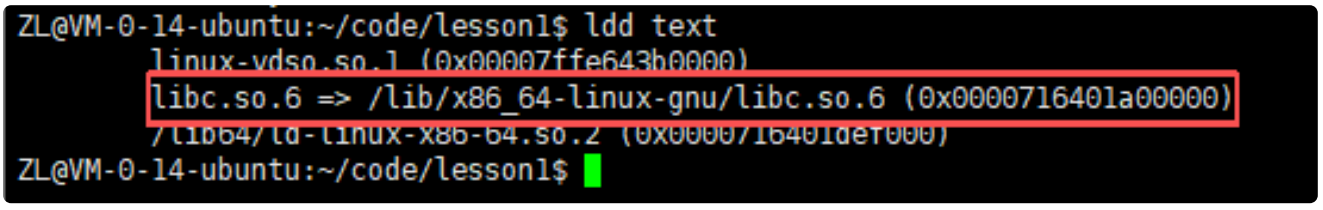
4.5.4 `ldd` 查看动态库依赖

```
ldd hello
```

通常会看到：

```
linux-vdso.so.1
libc.so.6 => /lib64/libc.so.6
/lib64/ld-linux-x86-64.so.2
```

这说明程序使用了 C 运行库和动态加载器。



4.5.5 动态库文件名

Linux 动态库常见形式：

```
libxxx.so
libxxx.so.6
libc.so.6
```

其中：

- `lib` 是库的常见前缀。
- `xxx` 是库名。
- `.so` 表示 shared object。
- 版本号用于区分 ABI 兼容级别。

链接时使用 `-lxxx`，通常会让链接器寻找 `libxxx.so` 或 `libxxx.a`。

4.6 强制静态链接

4.6.1 使用 `-static`

```
gcc hello.c -o hello-static -static
```

`-static` 告诉链接器尽量使用静态库。

4.6.2 为什么可能失败

很多系统默认只安装动态库，静态库需要额外安装。例如缺少 `libc.a` 时，静态链接会失败。

需要先确认静态库是否存在：

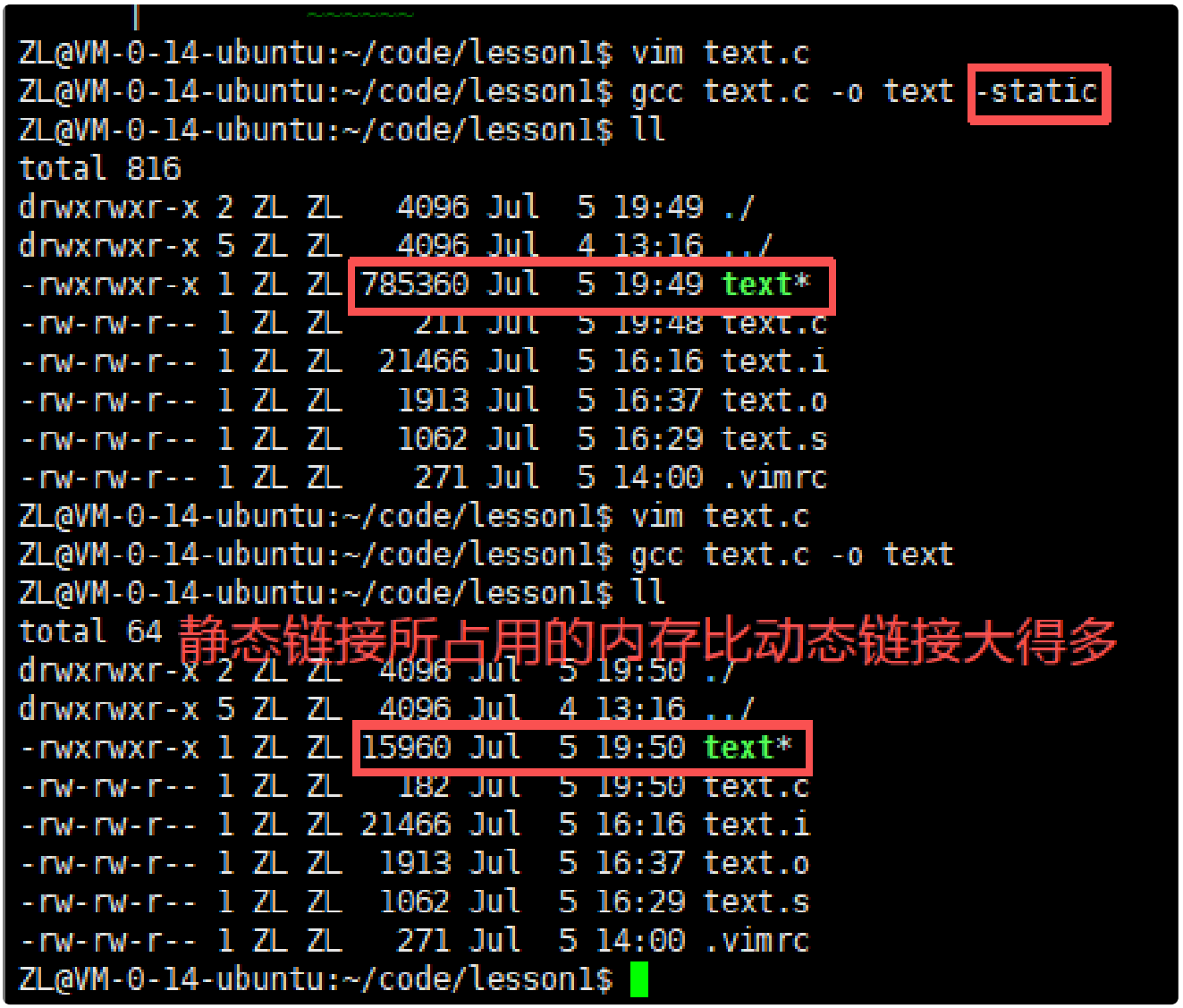
```
ls /usr/lib/libc.a
ls /usr/lib64/libc.a
```

不同发行版路径可能有所差异。

4.6.3 比较文件大小

```
ls -lh hello hello-static
```

通常会看到静态链接版本更大，因为它包含了程序实际使用的库代码。



4.6.4 验证静态链接结果

```
file hello-static
ldd hello-static
```

`file` 通常显示：

```
statically linked
```

`ldd` 可能显示：

```
not a dynamic executable
```

这说明程序没有以常规动态方式依赖共享库。

4.7 静态链接与动态链接总结

对比项	静态链接	动态链接
库代码是否进入可执行文件	需要的方法被合并进去	运行时加载共享库
常见库后缀	`.a`	`.so`
可执行文件大小	通常较大	通常较小
运行时依赖	较少	依赖对应动态库
库升级	通常需要重新链接	兼容时可直接替换共享库
部署	文件更自包含	需保证运行环境有正确库
gcc 常用选项	`-static`	默认行为

5. gcc 常用选项

5.1 阶段控制

5.1.1 预处理、编译、汇编

```
gcc -E hello.c -o hello.i
gcc -S hello.i -o hello.s
gcc -c hello.s -o hello.o
```

对应含义：

选项	停止位置
`-E`	预处理后停止
`-S`	生成汇编后停止
`-c`	生成目标文件后停止

5.1.2 输出文件

```
gcc hello.c -o hello
```

`-o` 用于指定输出名称。不指定时，完整编译通常会使用默认名称 `a.out`。

5.2 调试、警告与优化

5.2.1 调试信息

```
gcc hello.c -o hello -g
```

`-g` 生成调试信息，使调试工具能够关联源码行号和变量。发布程序不一定需要完整调试信息，但开发阶段非常有用。

5.2.2 显示警告

```
gcc hello.c -Wall -o hello
```

`-Wall` 打开常见警告。警告通常意味着：

- 隐式类型转换。
- 变量未使用。
- 参数不匹配。
- 可疑逻辑。

不要为了让编译“安静”而关闭所有警告。

5.2.3 优化级别

选项	含义
`-O0`	基本不优化，便于调试
`-O1`	基础优化
`-O2`	常用优化级别
`-O3`	更积极的优化

优化可能改变代码执行方式，影响调试时的变量观察和单步行为。开发阶段可优先使用 `-O0 -g`，发布阶段再评估优化选项。

5.3 链接选项

5.3.1 静态链接

```
gcc hello.c -o hello-static -static
```

尽量使用静态库。

5.3.2 生成共享库

```
gcc -shared source.o -o libdemo.so
```

`-shared` 用于生成共享库。实际项目中通常还要配合位置无关代码等选项，例如 `-fPIC`。

5.3.3 链接指定库

```
gcc main.o -L./lib -ldemo -o app
```

选项	作用
`-L目录`	增加库搜索目录
`-l库名`	链接名为 `lib库名.so` 或 `lib库名.a` 的库

5.3.4 增加头文件目录

```
gcc main.c -I./include -o app
```

`-I` 用于增加头文件搜索路径。

5.4 常用选项总表

选项	作用
`-E`	只预处理
`-S`	只编译到汇编
`-c`	编译到目标文件，不链接

选项	作用
`-o`	指定输出文件
`-g`	生成调试信息
`-Wall`	显示常见警告
`-O0` / `-O1` / `-O2` / `-O3`	设置优化级别
`-static`	强制使用静态链接
`-shared`	生成共享库
`-fPIC`	生成位置无关代码，常用于共享库
`-I目录`	增加头文件搜索路径
`-L目录`	增加库搜索路径
`-l库名`	链接指定库

6. gcc 实战命令

6.1 单文件一步编译

6.1.1 最简命令

```
gcc hello.c -o hello
```

6.1.2 开发阶段常用

```
gcc hello.c -Wall -g -o hello
```

含义：

```
-Wall   打开常见警告
-g      保留调试信息
-o      指定可执行文件名
```

6.2 单文件分阶段编译

```
gcc -E hello.c -o hello.i
gcc -S hello.i -o hello.s
```

```
gcc -c hello.s -o hello.o
gcc hello.o -o hello
```

执行：

```
./hello
```

6.3 多文件编译

6.3.1 分别生成目标文件

```
gcc -Wall -g -c main.c
gcc -Wall -g -c math.c
```

6.3.2 链接

```
gcc main.o math.o -o app
```

6.3.3 一次完成

```
gcc -Wall -g main.c math.c -o app
```

多文件分别编译的好处是定位错误更明确，也为后续增量构建做准备。不过自动化构建不属于本篇范围。

6.4 验证可执行文件

6.4.1 查看文件格式

```
file app
```

6.4.2 查看动态依赖

```
ldd app
```

6.4.3 生成并对比静态版本

```
gcc hello.c -o hello
gcc hello.c -o hello-static -static
ls -lh hello hello-static
file hello hello-static
```

如果静态库已经安装，通常可以看到静态版本明显更大。

6.5 C++ 程序

```
g++ main.cpp -o app
```

开发阶段：

```
g++ -Wall -g main.cpp -o app
```

分阶段查看：

```
g++ -S main.i -o main.s
g++ -c main.s -o main.o
g++ main.o -o app
```

如果 C++ 程序由多个文件组成，最后仍要使用 g++ 完成链接，以确保 C++ 标准库被正确加入。

7. 常见错误与排查

7.1 预处理错误

7.1.1 找不到头文件

典型提示：

```
fatal error: xxx.h: No such file or directory
```

可能原因：

- 头文件路径写错。
- 未安装开发包。
- 头文件不在系统搜索路径。

解决时可以先用 `-I` 添加项目头文件目录。如果系统缺少开发包，需要安装对应头文件。

7.1.2 宏未定义

条件编译分支可能因为宏定义不同而出现不同结果。查看预处理结果：

```
gcc -E source.c -o source.i
```

在 `.i` 中搜索宏名，可以确认条件编译是否按预期生效。

7.2 编译错误

7.2.1 语法错误

典型提示：

```
error: expected ';' before ...
```

应优先查看报错位置附近，因为真正的错误可能出现在上一行。

7.2.2 类型错误

编译阶段会检查类型是否符合语言规则。修复时应修改源代码，而不是通过关闭警告掩盖问题。

7.3 汇编错误

汇编阶段问题相对少见。如果手动修改过 `.s` 文件，可能遇到指令或格式错误。

更常见的情况是使用了不匹配的目标架构，例如为 x86-64 生成的目标文件被错误地交给 ARM 工具链处理。

7.4 链接错误

7.4.1 `undefined reference`

表示链接器知道有某个符号被引用，但找不到其实现。

常见原因：

- 忘记把某个 `.o` 加入链接。
- 确实没有实现某个函数。
- 函数声明和定义不一致。
- 忘记链接所需库。
- C/C++ 名称修饰不匹配。

7.4.2 编译通过但链接失败

这说明源文件本身可以转换成目标文件，问题多位于符号解析或库连接阶段。

可以把 `.o` 文件保留下来，分别执行：

```
gcc main.o -o app
```

根据错误信息判断缺失的符号和库。

7.4.3 缺少静态库

使用 `-static` 后失败，可能提示找不到 `libc.a` 等静态库。

应先确认安装环境是否提供静态库，再决定是否使用静态链接。不要为了强制通过而随意删除 `-static` 诊断信息。

7.5 运行错误

7.5.1 找不到动态库

可能出现：

```
error while loading shared libraries
```

这表示可执行程序可以启动到加载器阶段，但运行时找不到某个 `.so`。

可以检查：

```
ldd app
```

需要根据程序配置和部署环境处理库路径，不能只把库随便复制到不可控目录。

7.5.2 `.o` 不能直接运行

记住：

```
.o 是链接前的目标文件
不是最终可执行程序
```

需要完成链接后再运行。

7.5.3 架构不匹配

如果程序使用了不同 CPU 架构的库或目标文件，链接或运行可能失败。使用 `file` 检查目标文件架构。

7.6 排查顺序

```
预处理是否成功
→ 头文件是否找到
→ 编译是否通过
→ 汇编是否生成 .o
```

```
→ 链接是否找到全部符号和库
→ 运行时动态库是否存在
→ 架构和库版本是否匹配
```

8. gcc 命令速查与练习

8.1 四阶段速查

```
gcc -E hello.c -o hello.i
gcc -S hello.i -o hello.s
gcc -c hello.s -o hello.o
gcc hello.o -o hello
```

直接从 C 生成目标文件：

```
gcc -c hello.c -o hello.o
```

一步生成可执行程序：

```
gcc hello.c -o hello
```

8.2 链接与库速查

```
gcc hello.c -o hello
gcc hello.c -o hello-static -static
ldd hello
file hello
ls -lh hello hello-static
```

多文件：

```
gcc -c main.c
gcc -c math.c
gcc main.o math.o -o app
```

共享库：

```
gcc -fPIC -c demo.c -o demo.o
gcc -shared demo.o -o libdemo.so
```

8.3 常用选项速查

```
-E       只预处理
-S       只编译到汇编
-c       编译到目标文件
-o       指定输出
-g       调试信息
-Wall    常见警告
-O0      关闭优化
-O2      常用优化
-static  静态链接
-shared  共享库
-fPIC    位置无关代码
-I       头文件目录
-L       库目录
-l       链接库
```

8.4 建议练习

8.4.1 观察四个阶段

1. 编写一个包含 `#include <stdio.h>` 的 C 文件。
2. 使用 `-E` 生成 `.i`，观察头文件展开。
3. 使用 `-S` 生成 `.s`，观察汇编代码。
4. 使用 `-c` 生成 `.o`。
5. 尝试直接运行 `.o`，记录报错。
6. 完成链接并运行可执行文件。

8.4.2 比较链接方式

1. 用默认方式编译可执行文件。
2. 用 `file` 和 `ldd` 查看动态链接信息。
3. 检查系统是否有静态库。
4. 使用 `-static` 生成静态版本。
5. 比较两个文件大小和 `file` 输出。

6. 如果缺少静态库，记录错误并说明原因。

8.4.3 观察头文件保护

1. 编写一个头文件，分别使用 `#ifndef` 和 `#pragma once`。
2. 在多个源文件中包含它。
3. 使用 `-E` 观察头文件是否被重复展开。

8.4.4 制造链接错误

1. 声明一个函数并在 `main` 中调用。
2. 暂时不提供函数实现。
3. 编译到 `.o` 时观察是否通过。
4. 链接时观察 `undefined reference`。
5. 补充实现并重新链接。

8.5 gcc 学习主线

```
源程序
→ 预处理
→ 编译成汇编
→ 汇编成目标文件
→ 链接目标文件和库
→ 生成可执行程序
→ 用 file 和 ldd 检查链接方式
```

本文只整理 gcc/g++、编译阶段、链接方式和库的基础知识，不展开 Makefile、进度条、Git 或调试器。